Московский Физико–Технический Институт

Численные методы
Задание №9

ФИО: Лазарь Владислав Игоревич
Группа: Б01-207а
Почта: lazar.v@phystech.edu

Долгопрудный, 2026 г.

Введение

Задание посвящено оптимизации операции умножения матриц $C = A \times B$ для современных процессорных архитектур. В данной работе рассматриваются матрицы с размерами, кратными степеням двойки, что позволяет избежать накладных расходов на обработку краевых условий при векторизации.

В данной работе процесс оптимизации рассматривается итеративно. Реализация разделена на следующие этапы:

1. **Наивный алгоритм** — обычная реализация умножения строк матрицы A на столбцы матрицы B.

2. **Оптимизация доступа к памяти** — транспонирование матрицы B для обеспечения линейного прохода по памяти.

3. **Векторизация с использованием интринсиков AVX2** — прямое управление 256-битными регистрами процессора.

   Использование инструкций `_mm256_loadu_ps` и `_mm256_fmadd_ps` позволит обрабатывать 8 `float` элементов за одну итерацию и использовать блоки FMA.

4. **Регистровое блокирование** — использование векторных регистров в качестве аккумуляторов. В этой реализации подразумевается удержание промежуточных сумм в регистрах процессора на протяжении всего внутреннего цикла. Так мы сможем минимизировать количество инструкций `LOAD/STORE` в кэш или память.

5. **Блочное кэширование** (`Cache Tiling`) — разбиение больших матриц на блоки, размеры которых позволяют им полностью умещаться в кэш-памяти процессора (L1/L2). Так мы минимизируем количество промахов кэша.

1. Наивное умножение матриц

Классическое определение произведения матриц $A \in \mathbb{R}^{M \times K}$ и $B \in \mathbb{R}^{K \times N}$ строится на вычислении скалярного произведения строк левой матрицы на столбцы правой. Результирующий элемент $C_{i,j}$ определяется следующим образом:

$$C_{ij} = \sum_{k=1}^{K} A_{ik} \cdot B_{kj},$$

где $i = \overline{1, M}$ — индекс строки матрицы A; $j = \overline{1, N}$ — индекс столбца матрицы B; k — индекс суммирования.

Или в матричном виде:

$$\underbrace{\begin{pmatrix} a_{11} & \cdots & a_{1k} & \cdots & a_{1K} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ \boxed{a_{i1} \quad \cdots \quad a_{ik} \quad \cdots \quad a_{iK}} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{M1} & \cdots & a_{Mk} & \cdots & a_{MK} \end{pmatrix}}_{\text{матрица } A \in \mathbb{R}^{M \times K}} \times \underbrace{\begin{pmatrix} b_{11} & \cdots & \boxed{b_{1j}} & \cdots & b_{1N} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ b_{k1} & \cdots & b_{kj} & \cdots & b_{kN} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ b_{K1} & \cdots & b_{Kj} & \cdots & b_{KN} \end{pmatrix}}_{\text{матрица } B \in \mathbb{R}^{K \times N}} = \underbrace{\begin{pmatrix} c_{11} & \cdots & c_{1j} & \cdots & c_{1N} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ c_{i1} & \cdots & \boxed{c_{ij}} & \cdots & c_{iN} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ c_{M1} & \cdots & c_{Mj} & \cdots & c_{MN} \end{pmatrix}}_{\text{матрица } C \in \mathbb{R}^{M \times N}}$$

Программная реализация наивного умножения

Логику реализации можно понять из иллюстрации матричного умножения выше.

```
// int M, K, N = ...
void matmul_naive(const float *A, const float *B, float *C) {
  for (int i = 0; i < M; ++i) {      // Passing through the lines of A
    for (int j = 0; j < N; ++j) {    // Passing through the columns of B
      float part_sum = 0.0f;

      for (int k = 0; k < K; ++k) { // Scalar product
        part_sum += A[i * K + k] * B[k * N + j];
      }

      C[i * N + j] = part_sum;
    }
  }
}
```

Эффективно ли?

С точки зрения архитектуры процессора, данный алгоритм имеет большой недостаток — плохую локальность данных по столбцам. Во внутреннем цикле (по k) происходит обращение к элементам B_{kj}. При каждом шаге k мы переходим на новую строку матрицы B. Поскольку матрицы хранятся в памяти по строкам (`Row-major order`), каждый такой переход заставляет процессор прыгать через N элементов. Это приводит к высокой вероятности промаха кэша, невозможности эффективного предсказания данных.

2. Умножение матриц с оптимизацией доступа к памяти

Исходя из описанных проблем, перейдем к банальнейшему улучшению. **Математически** просто транспонируем матрицу B, тогда с точки зрения локальности данных все будет отлично, ведь теперь прежние столбцы матрицы B станут строками, и процессор не будет каждый раз прыгать через N элементов — данные теперь лежат линейно.

$$\underbrace{\begin{pmatrix} a_{11} & \cdots & a_{1k} & \cdots & a_{1K} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ \boxed{a_{i1} \quad \cdots \quad a_{ik} \quad \cdots \quad a_{iK}} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{M1} & \cdots & a_{Mk} & \cdots & a_{MK} \end{pmatrix}}_{\text{матрица } A \in \mathbb{R}^{M \times K}} \times \underbrace{\begin{pmatrix} b_{11} & \cdots & b_{k1} & \cdots & b_{K1} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ \boxed{b_{1j} \quad \cdots \quad b_{kj} \quad \cdots \quad b_{Kj}} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ b_{1N} & \cdots & b_{kN} & \cdots & b_{KN} \end{pmatrix}}_{\text{матрица } B^T \in \mathbb{R}^{N \times K}} = \underbrace{\begin{pmatrix} c_{11} & \cdots & c_{1j} & \cdots & c_{1N} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ c_{i1} & \cdots & \boxed{c_{ij}} & \cdots & c_{iN} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ c_{M1} & \cdots & c_{Mj} & \cdots & c_{MN} \end{pmatrix}}_{\text{матрица } C \in \mathbb{R}^{M \times N}}$$

Однако, **программно** это не очень умно, ведь нам каждый раз придется транспонировать матрицу B. Это трата ресурсов на пустом месте. Рассмотрим на простейшем примере умножения матриц $A \in \mathbb{R}^{2\times 2}$ и $B \in \mathbb{R}^{2\times 2}$ как можно избежать транспонирования и при этом воспользоваться свойством пространственной локальности данных:

$$\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \times \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix} = \begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix}$$

Здесь имеем:

$$c_{11} = a_{11}b_{11} + a_{12}b_{21}$$

$$c_{12} = a_{11}b_{12} + a_{12}b_{22}$$

Мы можем не сразу считать конкретный элемент c_{ij}, а частично накапливать его сумму:

- Будем фиксировать строку i матрицы A. В этой строке фиксируем элемент $a_{ik} \equiv a$.
- Далее будем пробегаться по строке j матрицы B. Тогда можно утверждать, что

$$c_{ij} = a \cdot b_{kj}$$

Тем самым, мы проходимся в матрице B по элементам построчно (по j; фиксация k была выше вместе с фиксацией элемента a_{ik}), а значит воспользовались пространственной локальностью. А также мы нигде не транспонировали матрицы. То есть воспользовались исключительно перестановкой циклов в коде.

Программная реализация оптимизированного умножения

```
// int M, K, N = ...
void matmul_optimized(const float *A, const float * B, float * C) {
  for (int i = 0; i < M; ++i) {
    for (int k = 0; k < K; ++k) {
      float fixed_a = A[i * K + k];

      for (int j = 0; j < N; ++j)
        C[i * N + j] += fixed_a * B[k * N + j];
    }
  }
}
```

3. Векторизация с использованием инструкций AVX2 и FMA

Предыдущая оптимизация позволила эффективно использовать кэш-память, обеспечив линейный доступ к данным. Однако на уровне вычислительного ядра мы все еще работаем в скалярном режиме. Одна инструкция обрабатывает одну пару чисел. Процессоры архитектуры `x86_64` поддерживают технологию `SIMD`, позволяющую выполнять одну операцию над целым вектором данных.

Для реализации этого подхода воспользуемся расширением `AVX2`. Основным инструментом здесь выступают 256-битные регистры (`YMM`), каждый из которых способен вместить 8 чисел типа `float`.

Вместо того чтобы в самом внутреннем цикле умножать `fixed_a` на один элемент B, мы будем:

1. Загружать `fixed_a` и размножать его во все 8 ячеек векторного регистра.
2. Далее загружать сразу 8 последовательных элементов матрицы B в другой регистр.
3. В итоге перемножать эти векторы и прибавлять результат к 8 элементам матрицы C за одну инструкцию.

`FMA` позволяет вычислить выражение `a·b+c` за один такт процессора без промежуточного округления.

Программная реализация с интринсиками

```
// int M, K, N = ...
void matmul_avx2(const float *A, const float *B, float *C) {
  for (int i = 0; i < M; ++i) {
    for (int k = 0; k < K; ++k) {
      // Load element A[i][k] and propagate it by the entire register
      __m256 vec_a = _mm256_set1_ps(A[i * K + k]);

      // Go through the row of the matrix B and C
      for (int j = 0; j < N; j += 8) {
        // Load 8 elements from B[k][j], ..., B[k][j+7]
        __m256 vec_b = _mm256_loadu_ps(&B[k * N + j]);

        // Load 8 elements from C[i][j], ..., C[i][j+7]
        __m256 vec_c = _mm256_loadu_ps(&C[i * N + j]);

        // C = A * B + C
        vec_c = _mm256_fmadd_ps(vec_a, vec_b, vec_c);

        // Saving the result back to the C
        _mm256_storeu_ps(&C[i * N + j], vec_c);
      }
    }
  }
}
```

4. Регистровое блокирование

В предыдущей реализациия мы векторизовали вычисления, но продолжали обращаться к памяти для записи результата матрицы C на каждой итерации по k. В данной реализации был применен метод регистрового блокирования.

Выделим сразу несколько векторных регистров (в данном случае 8 регистров `YMM`) для хранения промежуточных сумм блока элементов 4×16. Перед началом внутреннего цикла по k регистры обнуляются. Вся арифметика накопления происходит внутри процессора без обращения к кэшу или `RAM`. Загружая один вектор из матрицы B, мы используем его сразу для умножения на несколько разных элементов из матрицы A (находящихся в разных строках).

Программная реализация регистрового блокирования

```
// int M, K, N = ...
void matmul_multi_reg(const float *A, const float *B, float *C) {
  // Go through the rows of matrix A with step 4
  for (int i = 0; i < M; i += 4) {
    // Go through the cols of matrix B with step 16 (2 registers of 8
       floats)
    for (int j = 0; j < N; j += 16) {
      // 8 accumulators for the 4x16 block
      __m256 acc00 = _mm256_setzero_ps();
      __m256 acc01 = _mm256_setzero_ps();
      ....

      for (int k = 0; k < K; ++k) {
        // Load elements from A
        __m256 a0 = _mm256_set1_ps(A[(i + 0) * K + k]);
        __m256 a1 = _mm256_set1_ps(A[(i + 1) * K + k]);
        __m256 a2 = _mm256_set1_ps(A[(i + 2) * K + k]);
        __m256 a3 = _mm256_set1_ps(A[(i + 3) * K + k]);

        // Load 16 elements from B
        __m256 b0 = _mm256_load_ps(&B[k * N + j]);
        __m256 b1 = _mm256_load_ps(&B[k * N + j + 8]);

        // Multiply and accumulate the sum
        acc00 = _mm256_fmadd_ps(a0, b0, acc00);
        acc01 = _mm256_fmadd_ps(a0, b1, acc01);
        ....
      }

      // Upload 4 registers into memory
      _mm256_store_ps(&C[(i + 0) * N + j], acc00);
      _mm256_store_ps(&C[(i + 0) * N + j + 8], acc01);
      ....
    }
  }
}
```

5. Блочное кэширование

При увеличении размерности матриц (например, до 1024×1024) производительность начинает деградировать из-за ограниченного объема кэш-памяти. Когда строка матрицы перестает помещаться в кэш L1/L2, процессор вынужден постоянно подгружать данные из более медленной RAM, что приводит к росту задержек. Будем решать эту проблему путем разделения итерационного пространства матрицы на подблоки, которые гарантированно остаются в кэше на протяжении всех необходимых вычислений.

Программная реализация блочного кэширования

```
// int M, K, N = ...
int BLOCK_I, BLOCK_J, BLOCK_K = 32;
void matmul_tiled_reg(const float *A, const float *B, float *C) {
  for (int bi = 0; bi < M; bi += BLOCK_I) {
    for (int bk = 0; bk < K; bk += BLOCK_K) {
      for (int bj = 0; bj < N; bj += BLOCK_J) {
        for (int i = bi; i < bi + BLOCK_I && i < M; i += 4) {
          for (int j = bj; j < bj + BLOCK_J && j < N; j += 16) {
            __m256 acc00, acc01, acc10, acc11, acc20, acc21, acc30, acc31;
            acc00 = _mm256_load_ps(&C[(i + 0) * N + j]);
            acc01 = _mm256_load_ps(&C[(i + 0) * N + j + 8]);
            ....

            for (int k = bk; k < bk + BLOCK_K && k < K; ++k) {
              __m256 a0 = _mm256_set1_ps(A[(i + 0) * K + k]);
              __m256 a1 = _mm256_set1_ps(A[(i + 1) * K + k]);
              __m256 a2 = _mm256_set1_ps(A[(i + 2) * K + k]);
              __m256 a3 = _mm256_set1_ps(A[(i + 3) * K + k]);
              __m256 b0 = _mm256_load_ps(&B[k * N + j]);
              __m256 b1 = _mm256_load_ps(&B[k * N + j + 8]);
              acc00 = _mm256_fmadd_ps(a0, b0, acc00);
              acc01 = _mm256_fmadd_ps(a0, b1, acc01);
              ....
            }
            _mm256_store_ps(&C[(i + 0) * N + j], acc00);
            _mm256_store_ps(&C[(i + 0) * N + j + 8], acc01);
            ....
          }
        }
      }
    }
  }
}
```

Методология измерений

- **Язык и компилятор**: `C++17`, компилятор `g++` со следующими флагами оптимизаций:

  `-O2/-O3 -mavx2 -mfma -ffast-math`.

  Использование `-O2` выбрано осознанно, чтобы убедиться насколько сильно одна только современная автоматическая векторизация может оптимизировать базовую логику.

- **Метрика измерения**: использование регистра `TSC` через инструкцию `__rdtsc()`.

- **Контроль точности**: каждый результат сравнивается с эталонным значением `Naive MatMul` с допустимой погрешностью $\varepsilon = 10^{-3}$.

- **Процедура замера**: для минимизации влияния системных прерываний и каких-либо случайностей в системе каждый эксперимент проводиться какое-то количество раз.

  Введем понятие *среднеквадратичного отклонения* σ, которое будет показывать, насколько сильно отдельные замеры отличаются от среднего арифметического по всем замерам для данной конфигурации "метод+размер":

  $$\sigma = \frac{\text{разброс в тактах}}{\mu}.$$

  Итак, для каждой конфигурации будем производить n замеров времени, в каждом замере по k прогонов. На k прогонах найдем среднее арифметическое. Таким образом, мы получим n значений средних арифметических, от которых вновь возьмем среднее арифметическое. Значения n и k будут определяться эмпирически в зависимости от устраиваемого нас среднеквадратичного отклонения и на `-O2`, и на `-O3`.

  Будем считать, что нас устраивают такие значения $\{n,\ k\}$, при которых $\sigma \leqslant 5\%$.

Эксперименты запускались на платформе с архитектурой `AMD Zen 3`:

- Процессор: `AMD Ryzen 7 5800U` (8 ядер, 16 потоков)

- Микроархитектура: `Zen 3 (Cezanne)`.

- Операционная система: `Linux Ubuntu`.

- Тактовая частота: базовая 1.9 ГГц. Фиксировалась с помощью:

  ```
  cpupower frequency-set -min 1.9GHz -max 1.9GHz
  ```

- Имеющийся набор инструкций: `AVX`, `AVX2`, `FMA3`.

Иерархия кэш-памяти:

- `L1d`: 32 КБ на ядро (всего 256 КБ). Хватает на матрицу 32×32. Отсюда и возьмем размер для блочного кэширования.
- `L2`: 512 КБ на ядро (всего 4 МБ). Хватает на матрицу 256×256.
- `L3`: 16 МБ (общий на все ядра). Хватает на матрицу 1024×1024.

Абсолютными результатами в этой работе мы будем считать такты процессора.

Оценка количества замеров n и прогонов k для оценки $\sigma(n, k)$

Конфигурация матрицы	Метод	Среднее кол-во тактов	$\sigma_{\text{-O2}}$, %	Среднее кол-во тактов	$\sigma_{\text{-O3}}$, %
32x32x32	MatMul naive	57000	3.60	6992	4.43
n = 200	MatMul opt	73197	0.38	3010	0.59
k = 2000	AVX2 + FMA	13664	0.05	3570	0.26
	MatMul reg	2668	0.84	2644	0.49
	MatMul tiled	2694	0.31	2658	0.49
64x64x64	MatMul naive	633137	1.36	66982	3.08
n = 200	MatMul opt	306844	1.48	22301	3.68
k = 800	AVX2 + FMA	60523	0.07	28425	0.75
	MatMul reg	21627	0.09	21588	0.21
	MatMul tiled	22295	0.16	21837	0.47
128x128x128	MatMul naive	9410207	0.72	1148909	1.04
n = 100	MatMul opt	2777366	0.08	272757	0.40
k = 250	AVX2 + FMA	448961	0.38	288015	0.23
	MatMul reg	178447	0.27	177498	0.32
	MatMul tiled	176407	0.23	174656	0.49
256x256x256	MatMul naive	81650614	1.81	11896935	1.55
n = 50	MatMul opt	22286332	0.10	2272410	2.17
k = 250	AVX2 + FMA	3474124	1.30	2895051	0.85
	MatMul reg	1460305	1.30	1500998	1.69
	MatMul tiled	1539569	0.31	1511381	0.42
512x512x512	MatMul naive	1105755177	1.64	111069581	1.18
n = 50	MatMul opt	182412908	1.47	26396327	1.78
k = 10	AVX2 + FMA	33584890	4.45	26252926	0.99
	MatMul reg	15965818	2.54	15531724	3.80
	MatMul tiled	14446999	2.26	13607571	1.02
1024x1024x1024	MatMul naive	16537471455	0.74	1952049423	3.32
n = 25	MatMul opt	1462172494	1.69	215704608	4.70
k = 10	AVX2 + FMA	257684579	4.22	223658857	3.01
	MatMul reg	160375823	0.19	165474182	2.46
	MatMul tiled	136248161	0.23	138780253	2.25

`g++ -O3/-O2 -mavx2 -mfma -ffast-math`

Анализ результатов

- В первую очередь хотелось бы отметить эффективность компилятора на разных уровнях оптимизаций. Между методами `MatMul naive` и `MatMul opt` разница в оптимизациях `-O2` и `-O3` огромна.

  На размере 32×32 переход с `-O2` на `-O3` ускоряет выполнение почти в 8 раз (с 57000 до 6992 тактов). Делаем вывод, что `-O3` успешно применяет авто-векторизацию даже к самому простому коду.

  При `-O2` `MatMul opt` часто проигрывает `MatMul naive` (например, на 32×32 имеем 73197 против 57000), так как перестановка циклов без векторизации создает лишнюю нагрузку. Однако на `-O3` компилятор подхватывает правильный порядок доступа к памяти и выдает 3010 тактов, что в 24 раза быстрее, чем на `-O2`.

  Оптимизация `-O3` почти всюду лучше `-O2`. Лишь при размере 1024×1024 наблюдаются шумы в результатах из-за недостаточного количества экспериментов.

- Во-вторых, можно сделать очевидный вывод, что собственные оптимизации писать нужно. Даже самая агрессивная оптимизация компилятора (`-O3` `MatMul naive` или `MatMul opt`) все равно значительно медленнее собственных оптимизаций.

  На размере 1024×1024 метод `MatMul tiled` на `-O3` работает за 138 млн тактов, в то время как `MatMul naive` требует 1.95 млрд тактов.

  Ускорение ручного блочного кэширования относительно `-O3` составляет $\sim 14x$, а относительно базового `-O2` — $\sim 121x$.

- При сравнении методов `MatMul reg` и `MatMul tiled` можно сделать следующие выводы (для `-O2`):

  – При размерах 32×32 — 256×256 методы практически идентичны с перевесом в сторону `MatMul reg` в силу отсутствия накладных расходов на управление блоками.

  – При размерах 512×512 — 1024×1024 метод `MatMul tiled` (136 млн) уверенно обходит Reg (160 млн), что является весьма сильным выигрышем в производительности.

- Показатели среднеквадратичного отклонения $\sigma(n, k) \leqslant 5\%$ для всех методов, что подтверждают приличную воспроизводимость результатов и детерминированность предложенных оптимизаций, так как они попадают в $\mu \pm 2\sigma$.